CLI

Auto completion
https://docs.haskellstack.org/en/v1.9.3/shell_autocompletion/
Add to bashrc

```
eval "$(stack --bash-completion-script stack)"
```

Commands

```
## Basic
#
$ stack --help
$ stack --version
$ stack --verbosity debug ...        #
silent,error,warn,info,debug
$ stack --verbose ...                # debug
$ stack -v ...
$ stack --silent ...
$ stack --color never ...            # no color. Default
$ stack --color auto ...             # always,auto,never
... etc

# Directory
$ stack --stack-root /home/yui/.stack ...    # specify
root explicitly
$ stack --work-dir . ...                     # specify
package explicitly
$ stack --stack-yaml ./stack.yaml ...        # specify
stack.yaml explicitly
...

#
$ stack upgrade                  # upgrade Stack to
latest


## Query
#
# path: location of stack related paths
# e.g. location of ghc
# e.g. location of global .stack
$ stack path                # all

...
stack-root: /Volumes/Data/yuinishizawa/stack
project-root: /Volumes/Data/yuinishizawa/Projects/
playground/haskell/tmp
config-location: /Volumes/Data/yuinishizawa/Projects/
playground/haskell/tmp/stack.yaml
...
compiler-exe: /Volumes/Data/yuinishizawa/stack/programs/
x86_64-osx/ghc-9.2.3/bin/ghc-9.2.3
compiler-bin: /Volumes/Data/yuinishizawa/stack/programs/
x86_64-osx/ghc-9.2.3/bin
compiler-tools-bin: /Volumes/Data/yuinishizawa/stack/
```

```
compiler-tools/x86_64-osx/ghc-9.2.3/bin
local-bin: /Users/yuinishizawa/.local/bin
...
```

```
$ stack path --compiler-exe        # location of GHC
$ stack path --stack-root          # ~/.stack
$ stack path --project-root
$ stack path --compiler-exe
$ stack path --program


# ls: list content
$ stack ls snapshots            # All snapshots in
~/.stack/snapshots
```

```
0534f42a5712273474c1347e02efa08c9af2101867ed93857ce9c0fe
801a9441            e.g. ~/.stack/snapshots/x86_64-osx/
0534f42a571...
0809293076b8b7a984a2ece026872600d382ee7a64b18c05cf59d9ad
5969b902            e.g. ~/.stack/snapshots/x86_64-osx/
0809293076b...
...
```

```
$ stack ls tools                # Tools installed
globally
```

```
ghc-9.2.4
ghc-9.2.3
```

```
$ stack ls dependencies         # Dependencies installed
for package
```

```
array 0.5.4.0
base 4.16.3.0
...
```

```
## Initialization
#
$ stack new myproject
$ stack new --bare myproject        # use existing
directory
$ stack new --force ...               # Overwrite
existing files

# stack init: convert Cabal project to Stack project
$ stack init ...



## Build
#
```

```
# stack build: the main command
$ stack build
$ stack build --help
$ stack build --dry-run ...
$ stack build --exec 'echo hello' ...            # Run
command after build
$ stack build --only-dependencies ...
$ stack build --only-snapshot ...
$ stack build --file-watch ...                   # Watch
+ compile
... etc

# build target: i.e. what to compile
# By default, builds:
# - dependencies
# - library
# - executables
$ stack build
$ stack build myproject            # Same

# target by package
$ stack build myproject            # i.e. this package
$ stack build mtl                  # dependency
$ stack build mtl-2.2.2            # version

# target by component
$ stack build myproject
$ stack build myproject:lib          # library
$ stack build myproject:exe:mymain   # executable


# GHC options
$ stack build --ghc-options="-Wall -Werror" ...
# GHC override
$ stack build --pedantic ...
 # --ghc-options='-Wall -Werror'
$ stack build --fast ...
 # --ghc-options='-O0' i.e. no optimizations
$ stack build --trace ...
$ stack build --profile ...


# test: Compile and run tests
# Alias for: build --test
$ stack test ...
$ stack test myproject:test:main              # Specify
test to run
$ stack build --test ...                        # same
$ stack test --no-run-test                     # compile
only
$ stack test --test-arguments="--help"          # test
args
$ stack test --test-arguments="+RTS -N"        # RTS
test args
```

```
# install: install project on machine
# Alias for: build --copy-bins
$ stack install ...                     # install in
~/.local/bin
$ stack build --copy-bins ...            # Same


# haddock: Compile haddock
# Alias for: build --haddock
$ stack haddock ...
$ stack build --haddock ...                           #
same
$ stack haddock --no-haddock-deps ...
$ stack haddock --haddock-args "-o ./docs" ...       #
arguments

# benchmark: Compile and run benchmarks
# Alias for: build --bench
$ stack bench ...
$ stack build --bench ...


# stack clean: delete caches in ./.stack-work
#
$ stack clean               #
$ stack clean myproject     # Remove by package
$ stack clean --full         # Remove ./.stack-work
$ stack purge               # same



## Execution
#
# stack exec: execute shell command in a special
environment
# - GHC in PATH
# - package executables in PATH
#
# i.e. similar to Node.js's npx
$ stack exec ls             # Any shell command
$ stack exec ls -- -lt       # with arguments
$ stack exec bash           # Enter shell

# GHC
$ stack exec ghc               # Run ghc
$ stack exec -- ghc             # Same
$ stack ghc
$ stack runghc
$ stack exec -- which ghc       # Stack shell with GHC

e.g.
/home/yui/.stack/programs/x86_64-osx/ghc-9.2.4/bin/ghc


# GHCI
$ stack exec ghci              # OK. Raw GHCI i.e.
```

```
packages not loaded
$ stack repl                    # BETTER. Use repl
instead to preload packages (See below)

# Package exec: i.e. run exec built by Stack
$ stack exec -- myMain
$ stack exec -- which myMain    # Path
```

```
e.g.
/home/yui/myproject/.stack-work/install/x86_64-osx/
aa323cc34c9333ca25b6f245f9501d5e9af7413e514f74082285334e
75d8952a/9.2.4/bin/myMain
```

```
# stack repl: GHC REPL, but with the package loaded in
the REPL
#
$ stack repl
$ stack ghci
$ stack exec ghci                 # Same but raw i.e.
nothing loaded

# By component
$ stack repl myproject            # Whole
$ stack repl myproject:lib        # Library i.e. local
modules



## Misc
$ stack sdist
$ stack dot
$ stack eval
$ stack script
... etc
```